Algorithmes et Pensée Computationnelle

Logiciels système

Le but de cette séance est de vous permettre de comprendre le rôle d'un système d'exploitation, de logiciels système, d'interpréteurs, de compilateurs et de bibliothèques. Cette séance sera composée de deux parties : la première est consacrée à l'installation des outils de travail ; la seconde contient des questions à choix multiples en lien avec les notions abordées dans le cours.
Le code utilisé dans les questions est également disponible sur Moodle.

1 Installation des outils

Suivre le guide suivant pour installer les outils qui seront utilisés lors des prochaines séances de TP. Le guide d'installation des outils sur Windows se trouve à cette adresse Moodle et sur macOS à cette adresse Moodle.

2 Systèmes d'exploitation

Le système d'exploitation (OS - Operating System) est le logiciel le plus important de l'ordinateur car il est responsable de tous les autres programmes et de leurs priorités d'exécution, il organise aussi les interactions et l'orchestration des différents composants physiques de la machine.

L'OS contrôle l'accès au matériel physique et gère le(s) processeur(s), la mémoire, le stockage, la sécurité et les périphériques externes. Par exemple, l'OS identifie quel utilisateur est connecté (au moment d'entrer le mot de passe à l'allumage), il reconnaît quelles touches du clavier et de la souris sont pressées, il affiche les images à l'écran et il sauvegarde les fichiers sur les disques durs internes ou externes.

La plupart du temps, de nombreuses applications sont exécutées au même moment sur le même ordinateur. Elles ont besoin d'utiliser les mêmes composants de l'ordinateur (CPU, RAM, Storage). Cependant, supposant qu'il existe un seul coeur dans le CPU, ces différents composants ne peuvent faire qu'une seule chose à la fois (même s'ils la font très rapidement), c'est la raison pour laquelle l'OS orchestre chaque tâche de manière à ce que l'utilisateur ait l'impression que tout se déroule en même temps.

Question 1: (🕓 *2 minutes*) **QCM**
Lequel de ces logiciels **n**'est **pas** un système d'exploitation ?

1. Microsoft Office
2. Microsoft Windows 98
3. Unix BSD
4. Gentoo
5. Aucune de ces réponses n'est correcte.

Question 2: (🕓 *2 minutes*) Laquelle de ces affirmations **ne** correspond **pas** au rôle du noyau *Kernel* du système d'exploitation ?

1. Servir d'interface de communication entre les parties logicielle et matérielle.
2. Gérer les ressources physiques de l'ordinateur.
3. Fournir les mécanismes d'abstraction du matériel.
4. Interpréter les instructions et les convertir en binaire.
5. Aucune de ces réponses n'est correcte.

Question 3: (🕓 *2 minutes*)
Avec lequel de ces éléments le système d'exploitation n'a aucune interaction ?

1. Le clavier et la souris.
2. Le processeur.

3. La carte réseau.
4. La mémoire vive (RAM).
5. Aucune de ces réponses n'est correcte.

Question 4: (🕒 *2 minutes*) Lequel de ces systèmes d'exploitation **ne** permet **pas** d'exécuter plus d'un programme en même temps ?

1. Ubuntu
2. macOS
3. Windows XP
4. MS-DOS

Question 5: (🕒 *3 minutes*) Que fait la commande suivante : **ls** *(Linux/macOS)* / **dir** *(Windows)* ?

1. Affiche sous forme de liste uniquement l'ensemble des fichiers du disque.
2. Affiche sous forme de liste l'ensemble des processus en cours.
3. Retourne uniquement les dossiers du répertoire courant.
4. Affiche les dossiers et fichiers du répertoire courant.
5. Aucune de ces réponses n'est correcte.

Question 6: (🕒 *2 minutes*)
Parmi les tâches suivantes, laquelle ne relève **pas** du système de fichiers (*filesystem*) ?

1. L'organisation des dossiers.
2. L'écriture et la lecture du **contenu** des fichiers sur les disques.
3. La gestion des chemins d'accès aux dossiers et aux fichiers.
4. L'organisation des fichiers.
5. Aucune de ces réponses n'est correcte.

3 Interpréteurs et compilateurs

Les ordinateurs ne "comprennent" pas les langages de programmation, il faut passer par un programme qui va convertir le code écrit par un humain en instructions que l'ordinateur comprend (à base de 0 et de 1).
Les interpréteurs et les compilateurs servent à faire ce travail de traduction, ils transforment donc les langages de programmation qui sont faits pour être compris et écrits par des humains, vers des instructions compréhensibles pour des ordinateurs. La façon dont les interpréteurs et les compilateurs opèrent est différente et les deux approches apportent leur propre lot de bénéfices et d'inconvénients :

Compilateurs	**Interpréteurs**
Le programme est intégralement traduit à l'avance	Les instructions sont traduites à la volée une par une
Le compilateur trouve les erreurs à la compilation	L'interpréteur relève les erreurs pendant l'exécution du programme
Exécution plus rapide	Exécution plus lente
Code machine généré et optimisé à l'avance	Code machine généré à la volée et optimisé à chaque exécution
Ex : C, C++, Java, Scala, Rust, etc.	Ex : Bash, Python, JavaScript, etc.

⚠ Attention

Sur macOS, pour exécuter du code Python depuis le terminal (shell), on écrit `python3` alors que sur Windows, on écrit `python`. Vous pouvez aussi essayer d'écrire les deux et voir quelle commande fonctionne. Pour exécuter du code Java, on exécute (dans le terminal) d'abord la compilation en écrivant `javac Example.java` et ensuite `java Example [arguments séparés par espaces]`.

Question 7: (*10 minutes*) En utilisant l'invite de commandes (Terminal), créer un fichier contenant les instructions suivantes :

```
if __name__ == "__main__":
    print("Hello world")

```

Enregistrer le fichier sous `hello.py`. Utiliser l'interpréteur de Python pour exécuter le programme que vous venez de créer.

Question 8: (*10 minutes*) En utilisant l'invite de commandes (Terminal), exécutez le programme Python suivant en lui passant des paramètres.

```
import sys

if __name__ == "__main__":
    arguments = sys.argv
    if len(arguments) == 3:
        print(f"La somme de {sys.argv[1]} et {sys.argv[2]} est
    {int(sys.argv[1]) + int(sys.argv[2])}")
    else:
        print("Assurez-vous de passer deux arguments à votre
    programme")
```

Question 9: (*10 minutes*) Soit le programme suivant écrit en Java.

```
public class Question9 {
   public static void main(String args[]) {
      for(int i = 0; i<args.length; i++) {
         System.out.println("Paramètre[" + i + "]: " + args[i]);
      }
   }
}
```

Exécutez-le en utilisant l'invite de commandes (Terminal) en vous assurant d'y ajouter des paramètres.

4 Bibliothèques

Une bibliothèque est un ensemble de fonctions qui ont pour but d'être utilisées par d'autres programmes, mais une bibliothèque seule ne suffit pas pour faire un programme. Par exemple, que ce soit Google Chrome, Word, ou Instagram, presque tous les programmes ont besoin de manipuler des listes d'objets. Au lieu de réécrire l'ensemble des fonctions qui permettent de créer et de gérer des listes, ces différents programmes utilisent tous une bibliothèque écrite par un tiers qui offre toutes les fonctionnalités attendues d'une liste.

En Python, de nombreuses fonctions et structures de données sont disponibles dans la librairie standard de Python (`list`, `set`, `dict`, `str`, etc.) et de nombreuses autres bibliothèques peuvent être utilisées en utilisant le mot-clé `import`. Le but des exercices ci-dessous est de vous apprendre à lire la documentation d'un langage de programmation. Cela vous permettra d'être plus autonome pour la suite du cours.

Dans l'exemple suivant, nous importons la bibliothèque `math` qui propose de nombreuses fonctions mathématiques et nous essayons la fonction factorielle pour calculer `5!`.

>_Exemple

```
import math
fact = math.factorial(5)
print(fact)
```

Question 10: (🕒 *3 minutes*) À quoi sert la fonction `gcd(a,b)` de la bibliothèque `math` de Python ?

Question 11: (🕒 *3 minutes*) En Python et en Java, quelles fonctions utiliseriez-vous pour convertir un angle, dont la valeur est en radians, en degrés ?

Question 12: (🕒 *3 minutes*) En Python, à quoi sert la fonction `fsum()` et quels sont les paramètres à lui passer ?

Question 13: (🕒 *3 minutes*) En Python, est-ce que la bibliothèque `math` contient autre chose que des fonctions ? Si oui, quoi ?

Question 14: (🕒 *10 minutes*) Cherchez sur le net d'autres bibliothèques disponibles pour Python. Citez-en trois et ajoutez le lien vers leur documentation officielle.